

Attention Is All You Need

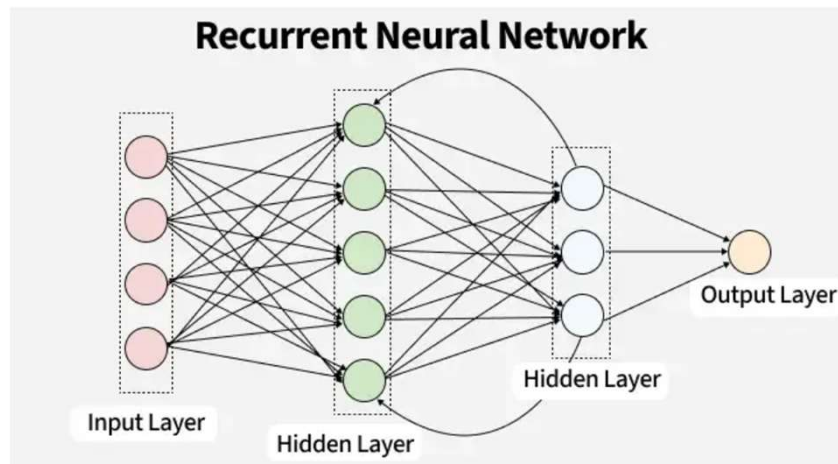
Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez,
Łukasz Kaiser, Illia Polosukhin

Reviewed by BeomSoo Son

Abstract

- Proposes the Transformer using only attention, without RNNs or CNNs.
- Achieves faster training and better translation performance.
- Also generalizes well to other sequence modeling tasks.

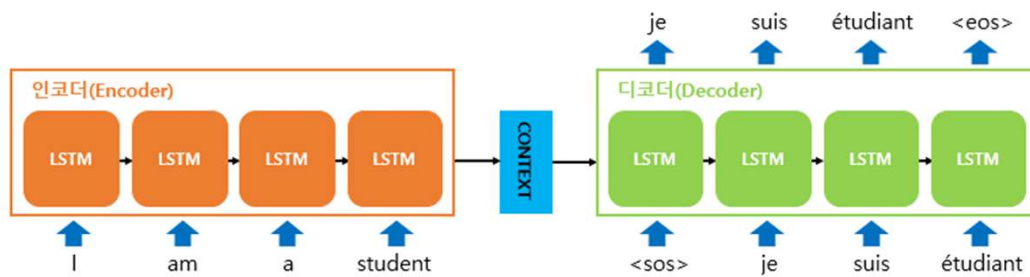
Sequence Modeling



- Recurrent models require high memory and computation.
- Sequential processing limits batching and memory efficiency.
- Each token mainly depends on the previous hidden state.

➔ Encoder-Decoder structured Seq2 Seq was proposed

Seq2 Seq



- The Encoder compresses the input sequence into a context vector.
- A fixed-size context vector creates an information bottleneck.

➔ Attention was introduced to overcome this limitation

Transformer: Model Architecture

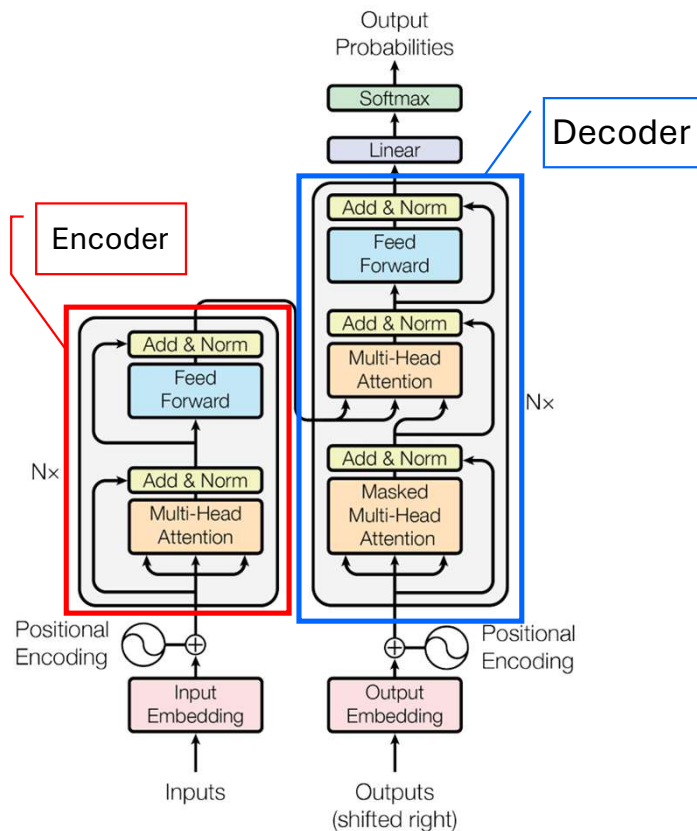
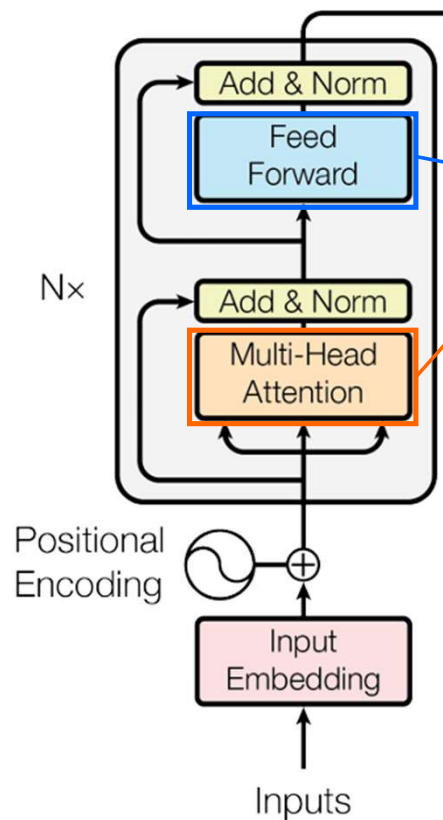


Figure 1: The Transformer - model architecture.

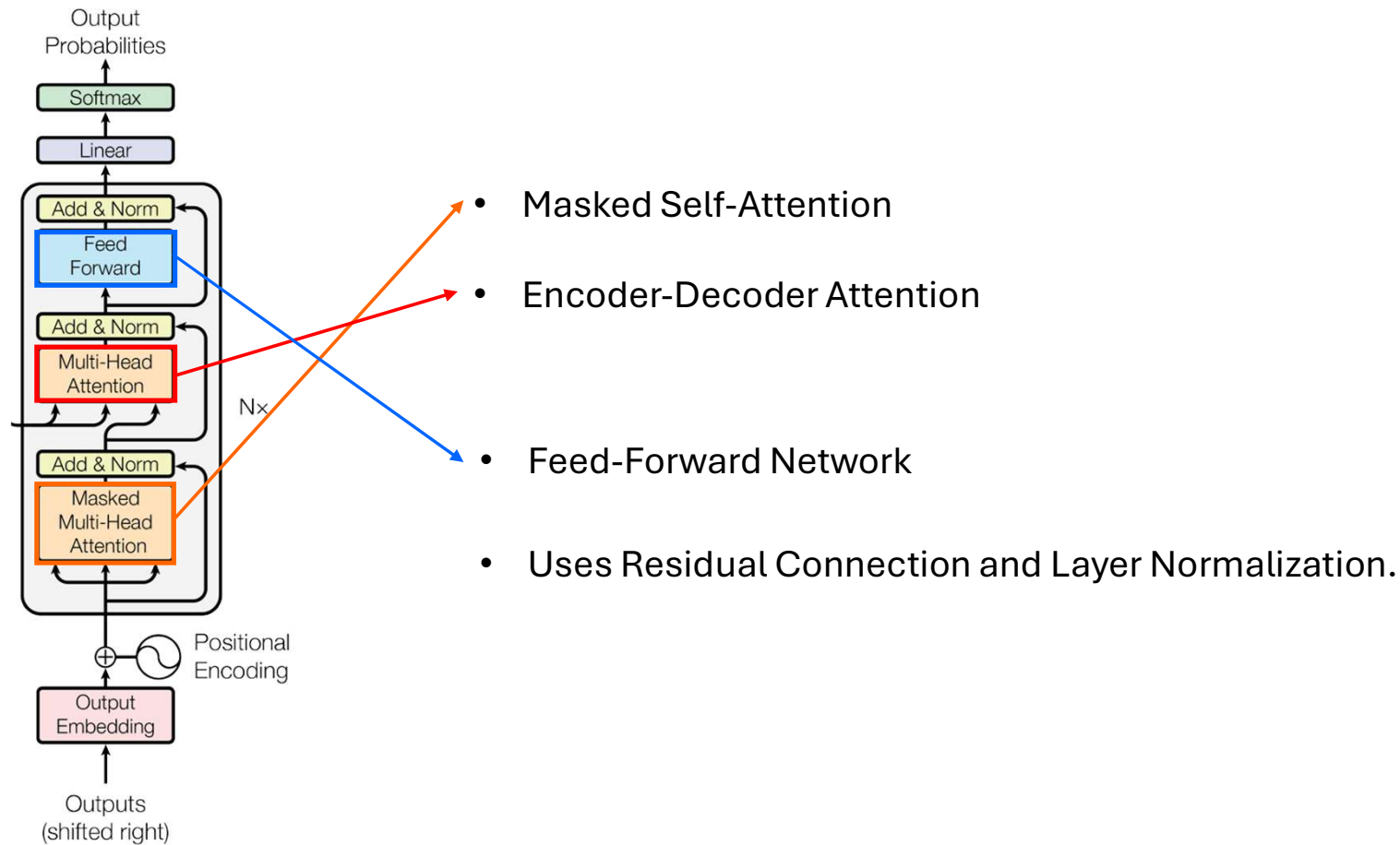
- Built by stacking N identical layers.
- Uses Input Embedding and Positional Encoding.
- Contains three types of attention.

Encoder



- Multi-Head Encoder Self-Attention
- Feed-Forward Network
- Uses Residual Connection and Layer Normalization.
- Stacks 6 identical layers.
- The model dimension is fixed at 512.
- The final encoder output is passed to the Decoder.

Decoder



Attention

- **Attention: relationship between words in a sentence**
- It uses **Query(Q)**, **Key(K)**, and **Value(V)**.

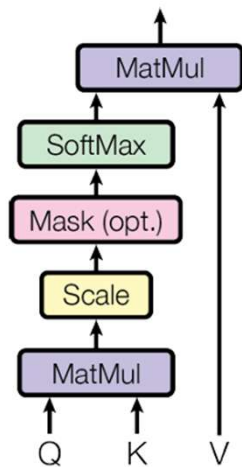
영어: I love her.

한국어: 나는 그녀를 사랑한다.

- 나는 이라는 주체가 Query
- 연관성을 찾는 대상인 (I, love, her)이 Key
- Q와 K의 유사성을 계산하여 유사한 만큼의 Value값

Scaled Dot-Product Attention

Scaled Dot-Product Attention

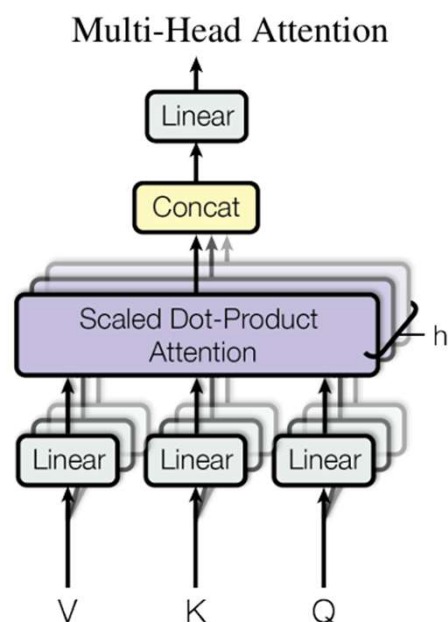


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q and K are used to compute attention scores.
- Scores are scaled by $\sqrt{d_k}$.
- Softmax converts scores into attention weights.
- The weights are applied to V.

- Represents a single attention operation.

Multi-Head Attention



- Splits attention into h parallel heads.
- Each head learns a different representation subspace.
- The outputs are concatenated and passed through a Linear Layer.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Why it is splited by h ?

- Multiple heads learn different representation subspaces and capture diverse dependencies.

Attentions

Which do you like better, coffee or tea?

- 문장 타입에 집중하는 어텐션

Which do you like better, coffee or tea?

- 명사에 집중하는 어텐션

Which do you like better, coffee or tea?

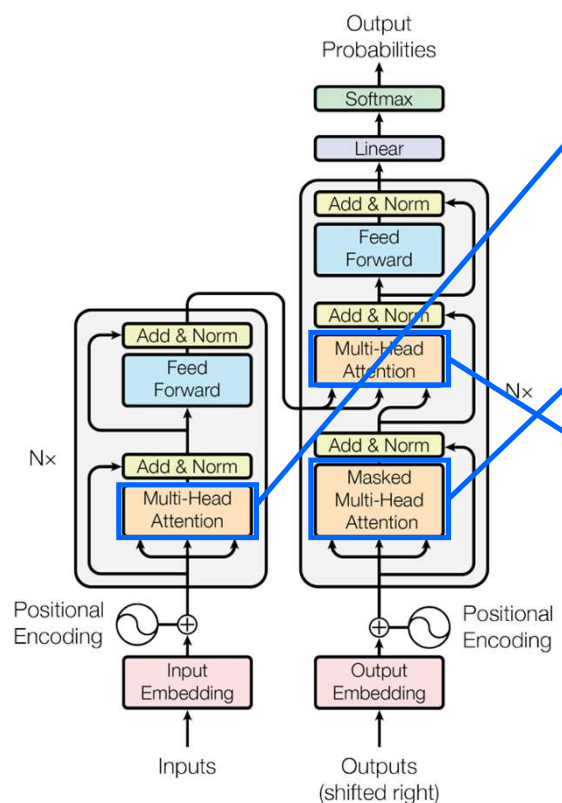
- 관계에 집중하는 어텐션

Which do you like better, coffee or tea?

- 강조에 집중하는 어텐션

- Different heads learn different attention patterns.
- The total computational cost remains similar to single-head attention.

Three Types of Attentions



- **Encoder Self-Attention:** Q, K, and V come from the Encoder.

- Each position can attend to all encoder positions.

Masked Decoder Self-Attention: The Decoder masks future tokens to preserve autoregressive behavior.

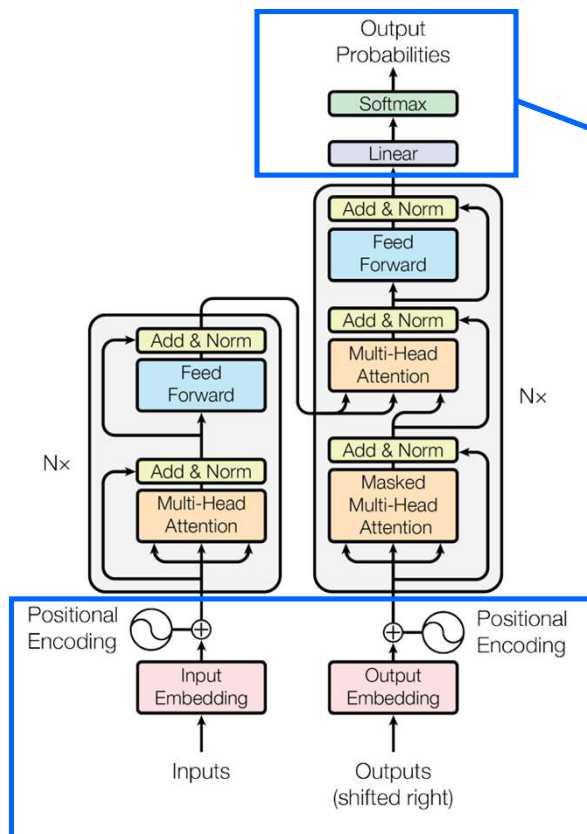
- Masked tokens are set to $-\infty$ to prevent attention.

- **Encoder-Decoder Attention:** Encoder-Decoder Attention uses the Decoder Query and the Encoder Key and Value.

- The Decoder learns relationships with the Encoder sequence.

Embeddings and Softmax

- Both the input and output embeddings go through an embedding layer, and the dimension used is $d_{model} = 512$
- The Decoder applies a Linear Transformation and Softmax to predict the next-token probabilities.
- The two embeddings share the same weight matrix with the Linear Transformation.



Positional Encoding

- The Transformer does not inherently model **sequential order** like RNNs or LSTMs.
- Attention alone does not contain sequence-order information.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

- $\sin(x)$ positional information is added in the same dimension as the embeddings.

$$\text{Patch Embedding} + \text{Position Embedding}$$

- Positional values between -1 and 1 are added to the Input and Output Embeddings.

Why Self-Attention?

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n: Sequence length, d: Representation dimension, k: kernel size

Complexity per Layer

- High-dimensional representations such as 256 or 512 are used to represent tokens effectively.
- Since sequence length is usually smaller than the representation dimension, Self-Attention has lower complexity.

Why Self-Attention?

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n: Sequence length, d: Representation dimension, k: kernel size

Sequential Operations

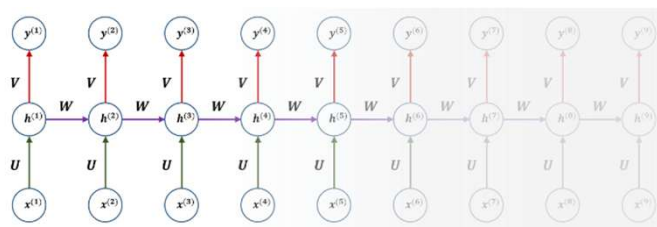
- RNNs require $O(n)$ sequential operations, while Self-Attention requires $O(1)$, making it more suitable for parallel processing.

Why Self-Attention?

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n : Sequence length, d : Representation dimension, k : kernel size

- **Path length between long-range dependencies:** Dependencies between distant positions in a sequence.



- Shorter path lengths make long-range dependencies easier to learn.
- RNNs have an $O(n)$ path length, while Self-Attention has $O(1)$ because each token can directly attend to all tokens.

Conclusion

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

- Achieved state-of-the-art performance on both WMT 2014 translation tasks with low training cost.
- Also generalized well to other tasks.

Conclusion

- Introduced the first Transformer architecture based entirely on attention.
- Replaced recurrent layers with Multi-Head Self-Attention.
- Achieved faster training and state-of-the-art translation performance.
- Inspired further research on attention-based models, including vision applications.

Thank You